AWS IAM — Complete Notes

Identity and Access Management · Who can do what on AWS

AWS IAM | Service Security | Scope Global | Cost Free

A reader-friendly, all-in-one reference for AWS IAM.

Table of Contents

---

1. What is IAM

AWS IAM controls **authentication** (who you are) and **authorization** (what you're allowed to do) for every AWS API call. It lets you create identities, attach **policies** that grant or deny permissions, and enforce least privilege across your account.

> 💡 **Mental model:** every AWS API call is a **request** that IAM evaluates against policies → **Allow** or **Deny**. **Default = implicit deny**; an explicit Allow is required, and an explicit Deny always wins.

Key facts: IAM is **global** (not region-scoped), **free**, and foundational to all AWS security.

---

2. 👤 Core Identities

Identity	What it is	Use
Root user	The account owner (email login)	Almost never — only a few root-only tasks. Lock it down with MFA.
IAM User	A long-lived identity for a person/app	Console password and/or access keys. Prefer roles where possible.
IAM Group	A collection of users	Attach policies once; users inherit. Cannot be nested or contain roles.
IAM Role	An identity with **temporary** credentials, assumed by trusted principals	EC2/Lambda/services, cross-account, federation. **Preferred** for workloads.

> ✅ Groups are for **humans**; roles are for **workloads and temporary access**. Don't bake long-lived access keys into apps — give them a **role**.

---

3. 📜 Policies & How They Work

A **policy** is a JSON document listing permissions. Types:

Policy type	Attached to	Purpose
Identity-based	User / group / role	What that identity can do. (Managed or inline.)
Resource-based	A resource (S3 bucket, SQS, KMS…)	Who (which principal) can act on it. Enables cross-account.
Permission boundary	User / role	The **max** permissions an identity can have.
SCP (Organizations)	OU / account	Guardrail capping permissions across accounts.
Session policy	Passed at AssumeRole	Further limits a session.

- **AWS managed** policies (maintained by AWS), **customer managed** (your reusable policies), or **inline** (embedded, 1:1 with an identity).

---

4. 🧰 Policy Document Structure

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowReadOneBucket",
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:ListBucket"],
      "Resource": [
        "arn:aws:s3:::my-bucket",
        "arn:aws:s3:::my-bucket/*"
      ],
      "Condition": {
        "IpAddress": { "aws:SourceIp": "203.0.113.0/24" }
      }
    }
  ]
}
```

Element	Meaning
Effect	`Allow` or `Deny`
Action	The API operations (`s3:GetObject`); `*` = all
Resource	ARNs the actions apply to
Principal	*(resource-based only)* who is granted access
Condition	Optional constraints (IP, MFA, tags, time, etc.)

---

5. ⚖️ How a Request is Evaluated

The decision flow for every request:

1. **Default deny** (nothing is allowed implicitly).
2. **Explicit Deny** anywhere → **DENY** (highest precedence).
3. **SCP** (if in an Org) must allow.
4. **Resource-based** policy or **identity-based** policy must allow.
5. **Permission boundary** and **session policy** must also allow (if present).
6. If allowed by all applicable policies and no explicit deny → **ALLOW**.

> 🔑 Remember: **an explicit `Deny` always overrides any `Allow`.** Permissions are the **intersection** of all guardrails (SCP ∩ boundary ∩ identity/resource policy).

---

6. 🎭 Roles & Temporary Credentials (STS)

- A **role** has a **trust policy** (who may assume it) + **permission policies** (what it can do).
- **AWS STS** issues **short-lived credentials** when a role is assumed (`AssumeRole` , `AssumeRoleWithWebIdentity` , `AssumeRoleWithSAML`).
- Uses: **EC2/Lambda instance roles**, **cross-account access**, **federated/SSO login**, **service-linked roles**.
- Temporary credentials **auto-expire** (minutes to hours) → far safer than static keys.

> ✅ The golden rule: **prefer roles over users with access keys** for anything programmatic. No secrets to rotate or leak.

---

7. 🚧 Permission Boundaries & SCPs

	Permission Boundary	Service Control Policy (SCP)
Applies to	An IAM user or role	An Org OU or account
Effect	Caps the identity's max permissions	Caps every principal in the account
Grants access?	❌ No — only limits	❌ No — only limits (even root is bound)
Use	Safe delegation (let teams create roles within a ceiling)	Org-wide guardrails (e.g. deny disabling CloudTrail)

> Neither **grants** permissions — they only set a **maximum**. You still need an identity/resource policy to actually allow an action.

---

8. 🔑 Authentication & MFA

- **Console:** username + password (+ MFA).
- **Programmatic:** access key ID + secret (or temporary STS creds).
- **MFA** (virtual app, hardware key, or FIDO2/passkey) adds a second factor — **enable it on root and all privileged users**.
- Enforce MFA via policy **Conditions** (`aws:MultiFactorAuthPresent`).

---

9. 🗝️ Access Keys & Credential Hygiene

- Access keys are long-lived secrets — **rotate regularly**, never commit to code/repos.
- Prefer **roles / IAM Identity Center / short-lived creds** over static keys.
- Use the **credential report** and **Access Analyzer** to find unused keys and over-broad access.
- Store secrets in **Secrets Manager** / **SSM Parameter Store**, not in source.

---

10. 🌐 IAM Identity Center & Federation

- **IAM Identity Center** (formerly AWS SSO) — central human sign-in across many accounts with **permission sets**; integrates with external IdPs (Okta, Entra ID, Google).
- **Federation** — trust an external IdP (SAML 2.0 / OIDC) so users log in with corporate credentials and assume roles; no IAM user per person.
- **Web identity federation** — mobile/web apps get temporary creds via Cognito or an OIDC provider.

> For multi-account orgs, **Identity Center + roles** is the modern standard — avoid creating IAM users per person.

---

11. 🔎 Auditing & Tools

Tool	Purpose
IAM Access Analyzer	Finds resources shared externally; validates & generates least-privilege policies.
Credential Report	CSV of all users, key age, MFA, last use.
Last Accessed (Access Advisor)	Shows which services an identity actually used → trim unused permissions.
CloudTrail	Records every API call for audit/forensics.
Policy Simulator	Test whether a policy allows/denies a given action.

---

12. ⌨️ Common CLI Commands

```
aws iam create-user --user-name alice
aws iam create-group --group-name developers
aws iam add-user-to-group --user-name alice --group-name developers
aws iam attach-group-policy --group-name developers \
  --policy-arn arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess

aws iam create-role --role-name app-role \
  --assume-role-policy-document file://trust.json
aws iam attach-role-policy --role-name app-role \
  --policy-arn arn:aws:iam::aws:policy/AmazonDynamoDBFullAccess

aws iam create-policy --policy-name my-policy --policy-document file://policy.json
aws iam list-users
aws iam get-account-authorization-details        # full dump
aws sts get-caller-identity                       # who am I?
aws sts assume-role --role-arn arn:... --role-session-name s1
```

---

13. ✅ Best Practices

- **Lock down the root user**: MFA, no access keys, use only for the rare root-only tasks.
- **Least privilege** — grant the minimum; widen only as needed (use Access Analyzer to refine).
- **Prefer roles** + temporary credentials over IAM users + static keys.
- **Enable MFA** everywhere; enforce it with conditions for sensitive actions.
- **Rotate** access keys; remove unused users/keys/permissions.
- Use **groups** to manage human permissions; **permission boundaries** for safe delegation; **SCPs** for org guardrails.
- **Centralize** sign-in with IAM Identity Center; **audit** with CloudTrail + credential reports.
- Never embed secrets in code — use **Secrets Manager** / **roles**.

---

14. 🧠 Quick Mental Model

- **IAM = who can do what.** Global, free, evaluated on every API call.
- Identities: **root** (avoid), **users** (humans), **groups** (bundle users), **roles** (workloads + temporary access — *prefer these*).
- **Policies** (JSON) grant/deny via **Effect + Action + Resource + Condition**.
- Evaluation: **default deny** → **explicit Deny wins** → **must be allowed by SCP ∩ boundary ∩ identity/resource policy**.
- **Roles + STS** give short-lived creds — no secrets to leak.
- **Boundaries & SCPs cap, never grant. MFA** + key rotation + least privilege are the daily disciplines.
- Audit with **Access Analyzer, credential report, CloudTrail**.

---

15. ❓ Common Interview Questions

Rapid-fire questions interviewers ask about IAM:

- **Q: How is an IAM request evaluated?** — Default deny → an explicit **Deny** always wins → must be allowed by SCP ∩ permission boundary ∩ identity/resource policy.
- **Q: User vs Role?** — User = long-lived identity (often with static keys). Role = assumed identity with **temporary** STS credentials — preferred for workloads and cross-account.
- **Q: Why prefer roles over access keys?** — Temporary credentials auto-expire and auto-rotate; nothing to leak or rotate manually.
- **Q: Permission boundary vs SCP?** — Both only **cap** permissions (never grant). Boundary limits one user/role; SCP limits every principal in an account/OU.
- **Q: Identity-based vs resource-based policy?** — Identity-based attaches to a user/role; resource-based attaches to a resource and names a **Principal** (enables cross-account).
- **Q: How does an EC2 instance get AWS permissions?** — Via an attached IAM role (instance profile) delivering temporary creds through the metadata service.
- **Q: Is IAM regional?** — No — IAM is a **global** service.
- **Q: How do you enforce MFA for sensitive actions?** — A policy condition on `aws:MultiFactorAuthPresent`.

Notes compiled as a quick reference for AWS IAM.